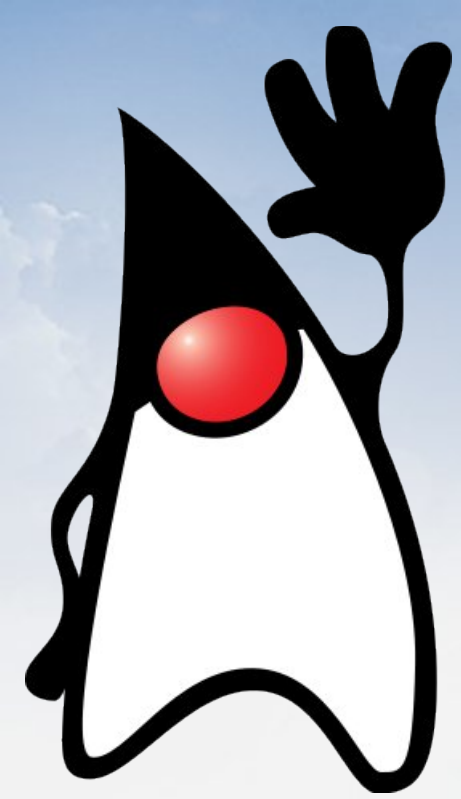


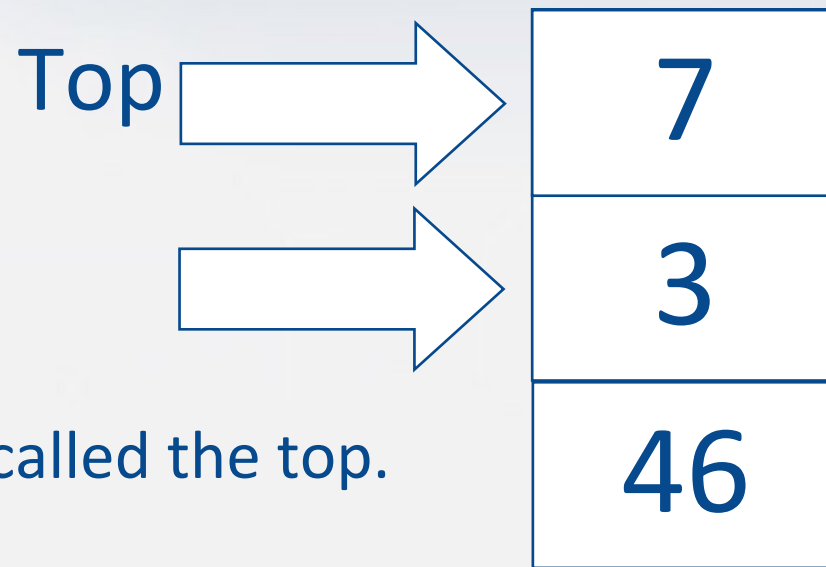


Stacks in Java

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

What is a Stack?

- A stack is a sequence of nodes in which we can only access the last item. We can either add to the top or remove from the top.
- Ex: Stack of dishes in the kitchen



The first node is called the top.
Stack uses LIFO.

Removing from stack is called pop

Adding to stack is called push

Getting the element without removal is called peek

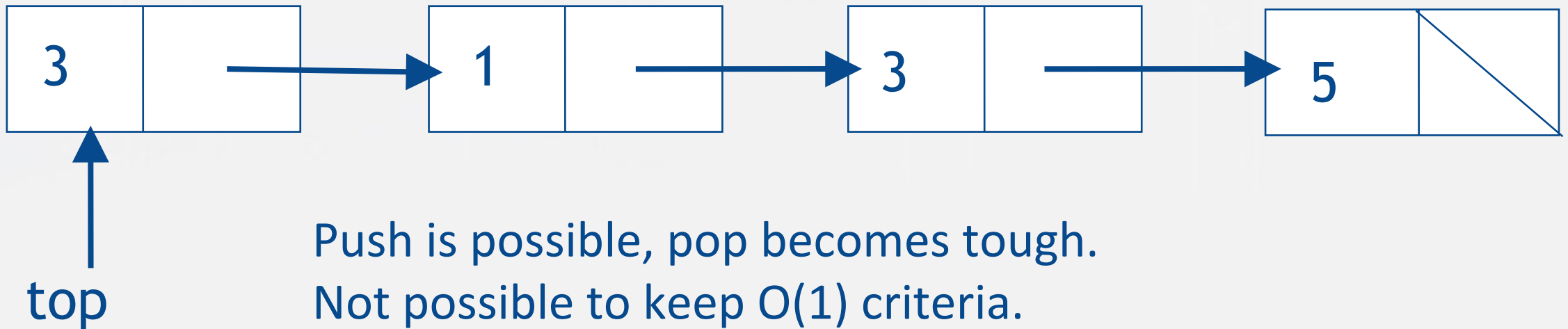
Implementation

- Push, pop and peek, all have $O(1)$ run time complexity
 - There is no traversal of elements in stack, as a standard operation
- Stack can be implemented with arrays and Linked Lists.
- What are the advantages and disadvantages of those implementations?
- Stack is used in
 - undo-redo
 - program memory/running
 - cases where last element/case has priority among others

Stack as a linked list

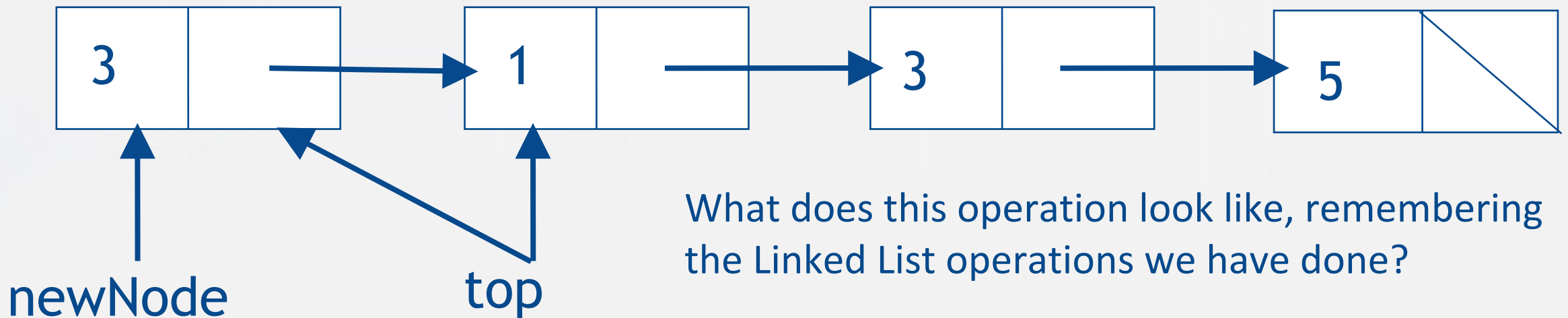
We push/pop the head of the linked list. What happens if we wanted to push/pop from the tail? If the arrows were reverse?

- Can you push?
- Can you pop?



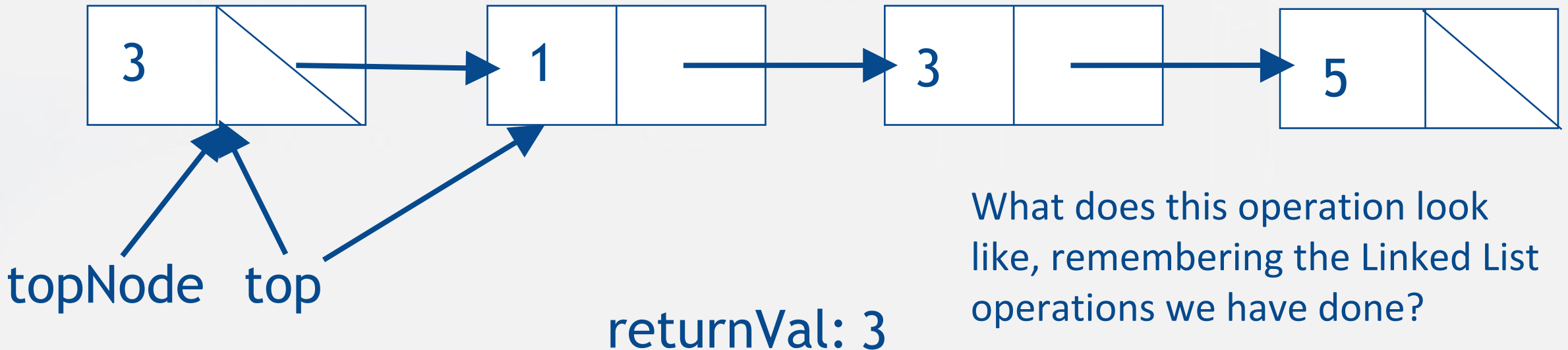
Push to Stack

```
Node newNode = new Node(3);  
newNode.setNext(top);  
top = newNode;
```



Pop from Stack

```
topNode = top; // topnode is what top shows  
returnVal = top.getItem(); // record the data  
top = top.getNext(); // UPDATE  
topNode.setNext(null); //optional
```



Push to Stack

```
top = 5; //size of the stack  
stackArr[top+1] = 9;  
top = top+1;
```

What happens if we filled all the positions of our starting array (stackArr)?

1	4	7	3	2	9
---	---	---	---	---	---

How can we get away from this problem? Will runtime change?

Pop from Stack

```
top = 6; // size is 6  
returnedElem = stackArr[top-1];  
top = top - 1;
```

We need to check if the stack is empty!

When we pop, what happens to 9?

1	4	7	3	2	9
---	---	---	---	---	---

What happens to 9 when we pop?

```
public int peek(){  
    return stackArr[top];  
}
```